

ProspectWeb — Feuille de route

Découpage de l'audit de due diligence en milestones et issues actionnables, du projet existant jusqu'à la levée de l'ensemble des objectifs.

7

MILESTONES

31

ISSUES

7

BLOQUANTS

Sommaire des milestones

La feuille de route reprend l'intégralité des constats de l'audit et les organise en 7 milestones séquencées, du plus urgent (stopper les coûts et restaurer la confiance) au plus structurel (scalabilité et différenciation). Chaque issue est autonome : elle peut être prise par n'importe quel contributeur à partir de sa seule description.

M0	Phase 0 — Stopper l'hémorragie & rétablir la confiance	6 issues · Jours
M1	Comptes, sécurité & contrôle des coûts	6 issues · Semaines
M2	Persistance & infrastructure scalable	3 issues · Semaines
M3	Fiabilité produit & expérience client	7 issues · Semaines
M4	Conformité juridique	2 issues · Semaines
M5	Qualité, tests & dette technique	3 issues · Continu
M6	Scalabilité, continuité & différenciation	4 issues · Trimestre

Grille de criticité : **Bloquant** empêche une mise en production ou commercialisation sérieuse — **Majeur** impact fort à corriger avant montée en charge — **Mineur** à traiter mais non urgent — **Cosmétique** finition.

Comment lire une issue. Chaque issue comporte un **Constat** (le problème vérifié dans le code, avec références `fichier:ligne`), un **Objectif** (l'état cible), une liste de **Tâches** concrètes, des **Critères d'acceptation** vérifiables, et les **Références** de fichiers concernés. Certaines issues notent leurs dépendances (ex. « dépend de PW-14 ») : suivre l'ordre des milestones les respecte naturellement.

Neutraliser sans délai les trois blocants à coût quasi nul de mise en œuvre : le robinet de coût Google, les données fabriquées et les mentions juridiques fausses. Aucune de ces issues ne nécessite de refonte ; ce sont des correctifs ciblés et urgents.

Bloquant ×4**Majeur** ×1**Mineur** ×1**PW-01 Liste blanche stricte et plafonnement du paramètre `types`****Bloquant****CONSTAT**

Le contrôleur découpe le paramètre `types` tel quel (`search.ts:62-64`) puis le service envoie chaque type fourni dans `fetchByTypeMultiArea` — jusqu'à 5 appels Google par type (`places.ts:173-180`). Aucun filtrage contre une liste blanche n'est appliqué. Fournir `types=t1,t2,...,t50` (même des types invalides, que Google facture quand même) déclenche ~250 appels Google payants pour une seule requête. Couplé à l'absence d'authentification, c'est un vecteur de déni de service financier.

OBJECTIF

Rendre impossible l'amplification du coût par le paramètre `types` : seuls des types réellement commerciaux sont acceptés, et leur nombre par requête est borné.

TÂCHES

- Rejeter avec un code `400` tout type absent de la constante `COMMERCIAL_TYPES` déjà définie dans `places.ts`.
- Plafonner le nombre de types par requête (ex. 8 à 10 maximum) et renvoyer une erreur claire au-delà.
- Valider côté contrôleur (`search.ts`) avant tout appel Google, pas seulement côté service.
- Documenter la liste des types autorisés dans la réponse d'erreur pour guider le client.

CRITÈRES D'ACCEPTATION

- ✓ Une requête contenant un type hors liste blanche est rejetée en `400` sans appel Google.
- ✓ Une requête avec plus de N types est rejetée avec un message explicite.
- ✓ Un test d'intégration couvre les deux cas de rejet.

RÉFÉRENCES : `backend/src/routes/search.ts:62-64` · `backend/src/services/places.ts:173-180`

PW-02 Plafonds de coût & alertes budgétaires Google Cloud

Bloquant

CONSTAT

Aucun plafond de requêtes par jour (QPD) ni budget n'est appliqué, ni dans le code ni (constaté comme angle mort) garanti dans la console Google. Google a supprimé le crédit universel de 200 \$/mois en mars 2025 au profit de plafonds gratuits par SKU bien plus petits. Combiné à l'application ouverte, rien ne borne la dépense : quelques milliers de requêtes nocturnes peuvent produire une facture de plusieurs centaines à milliers d'euros.

OBJECTIF

Garantir qu'aucun scénario d'usage ne puisse générer une facture incontrôlée, via une barrière défensive côté plateforme Google et un suivi applicatif.

TÂCHES

- Configurer des quotas QPD par SKU dans la console Google Cloud (Places, Geocoding).
- Mettre en place des alertes budgétaires (budget alerts) avec seuils et notifications e-mail.
- Restreindre la clé API Google (référents HTTP / IP serveur, APIs autorisées).
- Ajouter un compteur d'appels applicatif (journalisé) pour observer la consommation réelle.

CRITÈRES D'ACCEPTATION

- ✓ Des quotas QPD sont effectivement configurés et documentés.
- ✓ Une alerte budgétaire se déclenche en test à seuil bas.
- ✓ La clé API est restreinte et ne fonctionne pas hors des origines attendues.

RÉFÉRENCES : [backend/src/services/places.ts:5-16](#) · [Console Google Cloud \(hors dépôt\)](#)

PW-03 Supprimer toutes les données fabriquées de la fiche prospect

Bloquant

CONSTAT

La fiche détail affiche des informations codées en dur, identiques pour tous les établissements : horaires « Lun–Sam 7h–20h » ([DetailClient.tsx:111](#)), « Instagram présent (783 abonnés) » ([DetailClient.tsx:126](#)) et « Fiche Google Maps complète » ([DetailClient.tsx:127](#)). Un utilisateur qui prospecte sur de fausses informations perd toute confiance dans l'ensemble de l'outil dès qu'il en découvre une.

OBJECTIF

N'afficher que des données réellement issues de l'API. Toute donnée non disponible est explicitement marquée « Non renseigné » plutôt qu'inventée.

TÂCHES

- Retirer les blocs horaires, Instagram/réseaux sociaux et « fiche complète » codés en dur.
- Afficher « Non renseigné » (ou masquer le champ) lorsque la donnée n'existe pas dans le modèle.
- Vérifier qu'aucune autre valeur statique ne se fait passer pour une donnée réelle ailleurs dans l'UI.

CRITÈRES D'ACCEPTATION

- ✓ Aucune valeur affichée n'est identique entre deux établissements différents si elle prétend être propre à l'un d'eux.
- ✓ Tout champ sans source API affiche un état « Non renseigné » non trompeur.

RÉFÉRENCES : [frontend/app/results/\[id\]/components/DetailClient.tsx:111](#) · [DetailClient.tsx:126-127](#)

PW-04 Retirer la mention de sources fausses « Pages Jaunes · SIRENE »

Bloquant

CONSTAT

Le pied de page affiche « Données issues de Google Maps · Pages Jaunes · SIRENE » ([page.tsx:38](#)) alors que le code n'utilise ni Pages Jaunes ni SIRENE. C'est une affirmation de provenance trompeuse, avec en plus des implications de licence propres à ces sources.

OBJECTIF

Que toute mention de source affichée corresponde exactement aux sources réellement utilisées.

TÂCHES

- Retirer « Pages Jaunes » et « SIRENE » du pied de page.
- Ne conserver que les sources effectives (Google Maps/Places, OpenStreetMap pour l'autocomplétion).
- Auditer le reste de l'UI et du README pour d'autres mentions de sources non utilisées.

CRITÈRES D'ACCEPTATION

- ✓ Le pied de page ne mentionne plus que des sources réellement exploitées par le code.
- ✓ Aucune autre mention trompeuse de source ne subsiste dans l'application.

RÉFÉRENCES : [frontend/app/page.tsx:38](#)

PW-05 Remonter les erreurs Google au lieu de les avaler silencieusement

Majeur

CONSTAT

Le service Places masque ses erreurs : [places.ts:115](#) fait `if (!res.ok) return [];` et [places.ts:118](#) fait `catch { return []; }`. Un **403** (quota dépassé, facturation suspendue, clé invalide) ou un **429** Google est transformé en liste vide, indiscernable d'« aucun commerce dans la zone ». Ni l'utilisateur ni l'exploitant ne voient jamais la panne — le produit « marche » mais ne retourne plus rien.

OBJECTIF

Distinguer sans ambiguïté « 0 résultat » d'« échec fournisseur », jusque dans l'UI.

TÂCHES

- Journaliser [res.status](#) et le corps de la réponse en cas d'échec Google dans [places.ts](#).
- Propager un état d'erreur typé (échec fournisseur) distinct d'un résultat vide jusqu'au contrôleur.
- Afficher côté frontend une bannière dédiée « service indisponible » différente d'un « aucun résultat ».

CRITÈRES D'ACCEPTATION

- ✓ Un **403/429** Google produit une erreur visible, pas une liste vide.
- ✓ Le frontend distingue visuellement panne fournisseur et absence de résultat.

RÉFÉRENCES : [backend/src/services/places.ts:115](#) · [places.ts:118](#)

CONSTAT

Plusieurs handlers renvoient au client des détails internes : `search.ts:53,105` renvoient `(err as Error).message`, et `geocode.ts:20` y inclut le `status` et l'`error_message` de Google. Cela peut fuiter des informations internes (état de la clé, motifs Google).

OBJECTIF

Ne jamais exposer de détails internes au client ; conserver le détail complet côté serveur.

TÂCHES

- Remplacer les messages renvoyés au client par des libellés génériques (ex. « Erreur interne »).
- Journaliser le détail complet de l'erreur côté serveur (avec contexte).
- Vérifier l'ensemble des points de sortie d'erreur du backend.

CRITÈRES D'ACCEPTATION

- ✓ Aucune réponse d'erreur client ne contient de message Google brut ni de stack interne.
- ✓ Les logs serveur conservent le détail exploitable pour le debug.

RÉFÉRENCES : `backend/src/routes/search.ts:53,105` · `backend/src/services/geocode.ts:20`

Introduire l'identité utilisateur — préalable à toute commercialisation — et fermer les vecteurs de sécurité structurels (proxy ouvert, rate-limiting cosmétique, SSRF, en-têtes). C'est le cœur de la refonte backend.

Bloquant ×2**Majeur** ×3**Mineur** ×1**PW-07 Comptes utilisateurs & authentification de toute recherche****Bloquant****CONSTAT**

Le secret partagé `x-internal-secret` protège le backend des appels directs de tiers, mais pas le frontend public de l'abus. N'importe qui atteignant l'application Vercel peut lancer des recherches illimitées payées par la clé Google de l'exploitant. Il n'existe ni compte, ni login, ni attribution d'usage : l'application est de fait un proxy ouvert et anonyme vers Google Places.

OBJECTIF

Placer toute recherche derrière une authentification utilisateur. Aucune requête payante ne doit pouvoir partir sans un utilisateur identifié.

TÂCHES

- Choisir et intégrer une solution d'auth (compte e-mail/mot de passe ou fournisseur OAuth).
- Créer un modèle utilisateur persistant (dépend de PW-12, base de données).
- Protéger l'endpoint `/api/search` : rejeter toute requête non authentifiée.
- Adapter le frontend (pages protégées, gestion de session/token).
- Conserver le secret interne front↔back comme défense en profondeur.

CRITÈRES D'ACCEPTATION

- ✓ Aucune recherche ne peut être déclenchée sans utilisateur authentifié.
- ✓ Chaque appel Google est attribuable à un utilisateur identifié.

RÉFÉRENCES : `backend/src/middleware/auth.ts` · `frontend/app/results/page.tsx:16-19`

PW-08 Quota par utilisateur & métering d'usage

Bloquant

CONSTAT

Aucun quota par utilisateur ni métering n'existe. Même avec des comptes, sans limite individuelle, un compte unique peut générer une dépense incontrôlée. Le métering est aussi le préalable à toute répercussion de coût en SaaS.

OBJECTIF

Limiter et mesurer la consommation Google par utilisateur et par période.

TÂCHES

- Compter les appels Google (ou unités de coût) par utilisateur et par jour.
- Appliquer un quota configurable par utilisateur ; bloquer proprement au-delà avec un message clair.
- Exposer la consommation à l'utilisateur (et à l'exploitant) pour transparence.
- Stocker les compteurs de manière durable (dépend de PW-12).

CRITÈRES D'ACCEPTATION

- ✓ Un utilisateur atteignant son quota est bloqué sans appel Google supplémentaire.
- ✓ La consommation par utilisateur est consultable et exacte.

RÉFÉRENCES : [backend/src/services/places.ts:126-180](#)

PW-09 Rate-limiting durable et correctement clé

Majeur

CONSTAT

Le rate-limiter utilise un store en mémoire ([rateLimit.ts](#)) : limite par instance, remise à zéro à chaque redéploiement, non partagée si plus d'une instance. Surtout, il est posé sur le backend que seul le frontend appelle : tous les utilisateurs passent par les IP serveur de Vercel, donc le quota de 20/min se calcule sur des IP partagées — soit blocage injuste des utilisateurs légitimes, soit inefficacité totale. [trust proxy](#) n'est jamais configuré, donc derrière Railway [req.ip](#) est probablement l'IP du proxy.

OBJECTIF

Un rate-limiting fiable, partagé entre instances, calé sur l'identité utilisateur réelle.

TÂCHES

- Externaliser le store du rate-limiter (Redis) pour le partager entre instances (dépend de PW-13).
- Clé le limiteur sur l'identité utilisateur (PW-07), pas sur l'IP de Vercel.
- Configurer `app.set('trust proxy', 1)` pour un [req.ip](#) correct.
- Réviser les seuils en fonction de l'usage réel mesuré (PW-08).

CRITÈRES D'ACCEPTATION

- ✓ Le rate-limiting survit à un redéploiement et fonctionne avec plusieurs instances.
- ✓ Un utilisateur ne peut plus être bloqué à cause du trafic d'un autre.

RÉFÉRENCES : [backend/src/middleware/rateLimit.ts](#) · [backend/src/index.ts](#)

PW-10 Durcir websiteChecker contre la SSRF

Majeur

CONSTAT

websiteChecker.ts:22-26 fait un `fetch` avec `redirect: 'follow'` vers l'URL `websiteUri` d'un commerce. Un attaquant créant une fiche Google dont le site redirige vers une IP interne (métadonnées cloud `169.254.x.x`, réseau Railway) fait sonder le réseau interne par le backend. La lecture est limitée (50 Ko, booléens/dates) mais aucune liste blanche ni blocage des IP privées n'existe.

OBJECTIF

Empêcher le backend de servir de sonde vers des ressources internes.

TÂCHES

- Après résolution DNS, refuser les IP privées, loopback et link-local (`10/8`, `172.16/12`, `192.168/16`, `127/8`, `169.254/16`).
- Limiter (ou interdire) le suivi de redirections, et revérifier l'IP à chaque saut.
- Idéalement, router ces requêtes via un proxy de sortie cloisonné.
- Conserver le timeout et le plafond de lecture existants.

CRITÈRES D'ACCEPTATION

- ✓ Une URL résolvant vers une IP privée n'est jamais contactée par le backend.
- ✓ Les redirections vers des IP internes sont bloquées.

RÉFÉRENCES : `backend/src/services/websiteChecker.ts:22-26`

PW-11 En-têtes de sécurité (helmet + Next) & secret en temps constant

Mineur

CONSTAT

Aucun `helmet` côté Express ; `next.config.ts` est vide (aucun `headers()`) : ni CSP, ni HSTS, ni X-Frame-Options définis par l'application au-delà des défauts Vercel. Par ailleurs, `auth.ts:13` compare le secret avec `!=='` (non constant-temps), exposant une attaque par timing théorique.

OBJECTIF

Durcir la surface HTTP et la comparaison du secret par défense en profondeur.

TÂCHES

- Ajouter `helmet` au backend Express.
- Définir un bloc `headers()` dans `next.config.ts` (CSP, HSTS, X-Frame-Options).
- Remplacer la comparaison `!=='` du secret par une comparaison en temps constant.

CRITÈRES D'ACCEPTATION

- ✓ Les en-têtes de sécurité sont présents dans les réponses (vérifiables via les devtools).
- ✓ La comparaison du secret n'est plus dépendante du temps.

RÉFÉRENCES : `frontend/next.config.ts` · `backend/src/middleware/auth.ts:13`

CONSTAT

[SearchForm.tsx:70](#) appelle directement [nominatim.openstreetmap.org](#) depuis le navigateur, sans User-Agent identifiant ni e-mail de contact. Cela expose l'IP des utilisateurs et viole la politique d'usage d'OSM (usage raisonnable, faible volume, ≤ 1 req/s, UA identifiant). Un usage commercial expose à un bannissement d'IP.

OBJECTIF

Rendre l'usage de Nominatim conforme à sa politique et masquer l'IP des utilisateurs.

TÂCHES

- Déplacer l'appel d'autocomplétion vers le backend (nouvel endpoint proxy).
- Ajouter un User-Agent identifiant avec e-mail de contact, et respecter ≤ 1 req/s.
- Mettre en cache les résultats d'autocomplétion fréquents.
- À terme, évaluer un self-hébergement de Nominatim pour le volume commercial (voir PW-30).

CRITÈRES D'ACCEPTATION

- ✓ Le navigateur n'appelle plus directement Nominatim.
- ✓ Les requêtes Nominatim portent un User-Agent identifiant et respectent le débit autorisé.

RÉFÉRENCES : [frontend/app/components/SearchForm.tsx:70](#)

Remplacer l'état en mémoire (cache, rate-limiter) et le stockage navigateur (CRM en `localStorage`) par une infrastructure partagée et durable, condition d'une mise à l'échelle horizontale et du multi-appareil.

Majeur ×3

PW-13 Cache durable et partagé (Redis)

Majeur

CONSTAT

Le cache serveur est en mémoire (`cache.ts`, `Map`, TTL 10 min) : perdu à chaque redéploiement et non partagé entre instances. C'est la principale mitigation de coût Google, et c'est la plus fragile possible — la mise à l'échelle horizontale la casse.

OBJECTIF

Un cache externalisé, partagé entre instances et persistant aux redéploiements.

TÂCHES

- Provisionner un Redis managé (Railway ou autre).
- Remplacer l'implémentation `Map` de `cache.ts` par un client Redis, en gardant l'interface.
- Conserver/ajuster la stratégie de TTL ; documenter les clés de cache.
- Mesurer le taux de hit après bascule.

CRITÈRES D'ACCEPTATION

- ✓ Le cache survit à un redéploiement et est partagé entre toutes les instances.
- ✓ Le comportement fonctionnel (TTL, clés) est préservé.

RÉFÉRENCES : [backend/src/services/cache.ts](#)

PW-14 Base de données & persistance serveur du CRM

Majeur

CONSTAT

Il n'existe aucune base de données. Les données utilisateur (CRM, prospects, notes) vivent en `localStorage` navigateur (`useProspects.ts`) : perdues en changeant de navigateur ou d'appareil, non partageables en équipe, effacées par un simple nettoyage de cache. C'est aussi le préalable aux comptes (PW-07) et au métering (PW-08).

OBJECTIF

Persister durablement, côté serveur, les comptes, les prospects et les notes, par utilisateur.

TÂCHES

- Choisir et provisionner une base de données (ex. PostgreSQL managé).
- Définir le schéma : utilisateurs, prospects, statuts CRM, notes, compteurs d'usage.
- Exposer des endpoints CRUD authentifiés pour le CRM.
- Migrer le frontend de `localStorage` vers l'API serveur (avec migration douce des données locales si pertinent).

CRITÈRES D'ACCEPTATION

- ✓ Les prospects et notes d'un utilisateur sont retrouvés après changement d'appareil.
- ✓ Aucune donnée CRM critique ne dépend plus du `localStorage`.

RÉFÉRENCES : [frontend/app/hooks/useProspects.ts](#)

PW-15 Plafond de concurrence & file d'attente pour la vérification de sites

Majeur

CONSTAT

La vérification des sites lance N `fetch` parallèles par recherche (timeout 6 s) sans plafond de concurrence ni file d'attente. Quelques recherches « tous types » simultanées (nombreux établissements × `fetch HTTP`) peuvent saturer l'event-loop et la mémoire d'une instance Railway. Le backend est par ailleurs sans état hormis le cache et le rate-limiter.

OBJECTIF

Borner la charge induite par la vérification de sites pour préserver la stabilité sous charge.

TÂCHES

- Introduire un plafond de concurrence (pool) sur les `fetch` de vérification de sites.
- Mettre en file d'attente les vérifications au-delà du plafond.
- Conserver le timeout de 6 s et la limite de lecture.
- Vérifier le comportement sous charge (plusieurs recherches simultanées).

CRITÈRES D'ACCEPTATION

- ✓ Le nombre de `fetch` simultanés est borné et configurable.
- ✓ Des recherches simultanées « tous types » ne saturent plus une instance.

RÉFÉRENCES : [backend/src/services/websiteChecker.ts](#) · [backend/src/routes/search.ts](#)

Rétablir la confiance du client final : fiabiliser le signal central de qualification, câbler ou retirer les fonctions fantômes, et rendre le parcours honnête (états de chargement, résultats partiels signalés, liens partageables).

Majeur ×6

Mineur ×1

PW-16

Fiabiliser le signal de récence et séparer « injoignable » d'« obsolète »

Majeur

CONSTAT

Le cœur de la proposition de valeur repose sur une heuristique fragile. « Récent » se réduit à la simple présence de « 2025 »/« 2026 » dans les 50 premiers Ko ([websiteChecker.ts:103](#)) : un « © 2026 » en pied d'un site vétuste le classe « actif », et un bon site sans année visible est classé « obsolète » (:103-105). Pire, un site momentanément injoignable ou qui bloque le bot est étiqueté « Site obsolète » ([search.ts:25-26](#)) — on peut alors dire à un commerce que son site est vieux alors qu'il fonctionne très bien.

OBJECTIF

Remplacer l'heuristique binaire fragile par un faisceau d'indices pondéré, exposer un niveau de confiance, et ne jamais confondre indisponibilité et obsolescence.

TÂCHES

- Distinguer explicitement les états : [actif](#), [obsolète](#), [injoignable/inconnu](#), [sans site](#).
- Remplacer la détection « année dans le texte » par plusieurs indices pondérés (en-tête [Last-Modified](#), présence de sitemap, technologies détectées, etc.).
- Exposer un niveau de confiance au client (et l'expliquer dans l'UI).
- Couvrir la logique par des tests unitaires (voir PW-26).

CRITÈRES D'ACCEPTATION

- ✓ Un site injoignable n'est jamais classé « obsolète ».
- ✓ Le classement repose sur plusieurs indices et affiche un niveau de confiance.

RÉFÉRENCES : [backend/src/services/websiteChecker.ts:103-105](#) · [backend/src/routes/search.ts:25-26](#)

PW-17 Câbler ou retirer les fonctions fantômes (filtres rapides, bouton e-mail)

Majeur

CONSTAT

Plusieurs fonctions sont affichées mais inopérantes. Les 5 « filtres rapides » de l'accueil ([SearchForm.tsx:27-33](#)) ne sont jamais ajoutés aux paramètres dans [handleSubmit \(:108-116\)](#) — ils ne font rien, et le backend n'a aucune notion de « réseaux sociaux » ou « récemment ouvert ». Le bouton « Envoyer un email » ([DetailClient.tsx:209-214](#)) n'a ni `href` ni `onClick`, et aucune donnée e-mail n'existe dans le modèle.

OBJECTIF

Tout élément d'interface visible doit avoir un effet réel, ou être retiré.

TÂCHES

- Pour chaque filtre rapide : soit l'implémenter de bout en bout (frontend → paramètres → backend), soit le masquer.
- Câbler le bouton e-mail à une donnée réelle (si une source d'e-mail existe via enrichissement), sinon le retirer.
- Vérifier qu'aucun autre contrôle « décoratif » ne subsiste.

CRITÈRES D'ACCEPTATION

- ✓ Chaque filtre affiché modifie réellement les résultats, ou n'est plus affiché.
- ✓ Le bouton e-mail agit, ou est supprimé.

RÉFÉRENCES : [frontend/app/components/SearchForm.tsx:27-33,108-116](#) · [frontend/app/results/\[id\]/components/DetailClient.tsx:209-214](#)

PW-18 Persister le statut CRM et les notes de la fiche détail

Majeur

CONSTAT

Sur la fiche détail, le statut et les notes sont stockés en `useState` local ([DetailClient.tsx:56-57](#)), déconnectés de `useProspects/persistence` : tout est perdu à la navigation. La page « Mes prospects » persiste, elle, ce qui crée une incohérence déroutante entre les deux écrans.

OBJECTIF

Aligner la fiche détail sur la persistance du CRM, sans perte à la navigation.

TÂCHES

- Brancher statut et notes de la fiche sur la couche de persistance du CRM (API serveur — dépend de PW-14).
- Garantir la cohérence d'affichage entre la fiche détail et la liste « Mes prospects ».
- Gérer les états de sauvegarde (en cours / enregistré / erreur).

CRITÈRES D'ACCEPTATION

- ✓ Statut et notes saisis sur la fiche sont conservés après navigation et changement d'appareil.
- ✓ La fiche et la liste affichent les mêmes valeurs pour un même prospect.

RÉFÉRENCES : [frontend/app/results/\[id\]/components/DetailClient.tsx:56-57](#) · [frontend/app/hooks/useProspects.ts](#)

PW-19 Rendre les liens profonds de fiche robustes et partageables

Majeur

CONSTAT

La fiche `/results/[id]/page.tsx` lit `sessionStorage('pw_search_results')`. Ouvrir un résultat dans un nouvel onglet, ou après expiration de la session, affiche « Commerce introuvable » : impossible de partager le lien d'un prospect.

OBJECTIF

Une fiche prospect doit se charger à partir de son seul identifiant, sans dépendre de la session.

TÂCHES

- Recharger la fiche depuis le backend par `id` lorsque la session est absente.
- Définir une stratégie de récupération (cache serveur, ou re-requête Places ciblée par Place ID).
- Gérer proprement le cas d'un id réellement introuvable.

CRITÈRES D'ACCEPTATION

- ✓ Un lien de fiche ouvert dans un nouvel onglet ou une nouvelle session affiche la fiche.
- ✓ Le partage d'un lien de prospect fonctionne.

RÉFÉRENCES : `frontend/app/results/[id]/page.tsx`

PW-20 Vrais états de chargement sur la page de résultats

Majeur

CONSTAT

La page résultats est un Server Component qui bloque jusqu'à obtention des données. La recherche peut durer plusieurs secondes (5×N appels Google + vérifications de sites à timeout 6 s, en `Promise.all`). Après le clic, l'utilisateur reste sur une page figée sans indicateur de progression — le spinner du bouton d'accueil disparaît à la navigation.

OBJECTIF

Donner un retour visuel continu pendant l'attente, supprimer l'impression de page figée.

TÂCHES

- Introduire un état de chargement réel (streaming Next et/ou skeleton) sur la page de résultats.
- Afficher un indicateur de progression pendant la recherche.
- Gérer l'affichage des erreurs (lien avec PW-05) dans le même flux.

CRITÈRES D'ACCEPTATION

- ✓ Après le clic de recherche, un état de chargement visible apparaît immédiatement.
- ✓ L'utilisateur n'est jamais face à une page figée sans retour.

RÉFÉRENCES : `frontend/app/results/page.tsx` · `frontend/app/results/components/ResultsClient.tsx`

PW-21 Signaler explicitement les résultats partiels ou plafonnés

Majeur

CONSTAT

Sans type sélectionné, le backend fait un appel par type depuis le centre unique (`places.ts:180`, branche `useMulti=false`), plafonné à ~20 résultats par type et sans densification multi-zones. En zone dense, des commerces sont manqués sans aucun avertissement « résultats incomplets ».

OBJECTIF

Quand les résultats sont plafonnés ou partiels, l'utilisateur doit en être informé clairement.

TÂCHES

- Détecter les cas de plafonnement (atteinte de la limite par type, zone dense).
- Remonter un indicateur « résultats potentiellement incomplets » jusqu'au frontend.
- Afficher un message explicite et, si pertinent, suggérer d'affiner la zone/les types.

CRITÈRES D'ACCEPTATION

- ✓ Un résultat plafonné est signalé visuellement à l'utilisateur.
- ✓ Le message indique comment obtenir une couverture plus complète.

RÉFÉRENCES : `backend/src/services/places.ts:180`

PW-22 Harmoniser les libellés de types & corriger l'accessibilité

Mineur

CONSTAT

Les tables de libellés/icônes (`ResultsClient.tsx:28-37`, `DetailClient.tsx`) référencent des types absents de `COMMERCIAL_TYPES` (ex. `laundry`, `locksmith`, `books`) tandis que des types réellement interrogés (`hotel`, `clothing_store`, `plumber...`) n'ont parfois ni libellé ni icône, d'où des chaînes brutes (`car_repair` → « car repair »). Côté accessibilité, beaucoup d'icônes SVG décoratives ne sont pas masquées aux lecteurs d'écran et la carte encode le statut par la seule couleur.

OBJECTIF

Aligner les libellés sur les types réellement utilisés et corriger les défauts d'accessibilité de base.

TÂCHES

- Synchroniser les tables de libellés/icônes avec `COMMERCIAL_TYPES` (un libellé et une icône par type interrogé).
- Retirer les références aux types non interrogés.
- Masquer les icônes décoratives aux lecteurs d'écran (`aria-hidden`).
- Ajouter un second canal au statut sur la carte (forme/libellé en plus de la couleur).

CRITÈRES D'ACCEPTATION

- ✓ Aucun type interrogé n'affiche de chaîne brute ou d'icône générique non voulue.
- ✓ Le statut est perceptible sans distinction de couleur ; les icônes décoratives sont ignorées par les lecteurs d'écran.

RÉFÉRENCES : `frontend/app/results/components/ResultsClient.tsx:28-37` · `backend/src/services/places.ts` (`COMMERCIAL_TYPES`)

Mettre l'usage des données tierces en conformité — conditions Google Places et RGPD — avant toute commercialisation. Les mentions de sources fausses sont déjà traitées en M0 (PW-04).

Bloquant ×1

Majeur ×1

PW-23

Conformité aux conditions de Google Places (stockage & export)

Bloquant

CONSTAT

L'export CSV et le stockage durable de noms, téléphones et adresses ([useProspects.ts](#)) vont au-delà du cache limité généralement autorisé : en règle générale, seul le [Place ID](#) peut être conservé durablement, et la réutilisation des données hors du contexte d'une carte Google est restreinte. Or le cœur d'usage — exporter pour démarcher — est précisément ce que les conditions encadrent.

OBJECTIF

Aligner le stockage et l'export sur ce que les conditions Google Places autorisent réellement.

TÂCHES

- Revoir précisément les conditions Google Places en vigueur (champs conservables, durée, réutilisation).
- Ne persister durablement que ce qui est permis (typiquement le [Place ID](#)), et re-résoudre les détails à la demande.
- Adapter l'export CSV en conséquence.
- Documenter la base de conformité retenue.

CRITÈRES D'ACCEPTATION

- ✓ Le stockage durable se limite aux champs autorisés par les conditions.
- ✓ L'export respecte les restrictions de réutilisation applicables.

RÉFÉRENCES : [frontend/app/hooks/useProspects.ts](#)

CONSTAT

Les téléphones d'indépendants et de professionnels constituent des données personnelles potentielles. L'outil ne propose ni base légale, ni opt-out, ni durée de conservation (données indéfiniment en `localStorage`), ni mention CNIL / politique de confidentialité.

OBJECTIF

Encadrer le traitement de données personnelles dans un cadre RGPD défendable.

TÂCHES

- Définir et documenter la base légale du traitement de prospection.
- Mettre en place une durée de conservation et une purge automatique.
- Prévoir un mécanisme d'opt-out / suppression sur demande.
- Rédiger et publier une politique de confidentialité (et mentions CNIL le cas échéant).

CRITÈRES D'ACCEPTATION

- ✓ Une base légale et une durée de conservation sont définies et appliquées techniquement.
- ✓ Un opt-out / une suppression sur demande est possible ; une politique de confidentialité est publiée.

RÉFÉRENCES : [frontend/app/hooks/useProspects.ts](#)

Rendre le projet maintenable et sûr à faire évoluer : introduire des tests sur la logique critique, corriger la documentation trompeuse qui casse l'installation, et supprimer le code mort.

Majeur ×2

Mineur ×1

PW-25 Suite de tests sur la logique critique + couverture en CI

Majeur

CONSTAT

Il n'existe aucun test (zéro fichier `.test/.spec`). La CI (`.github/workflows/ci.yml`) n'exécute que `lint`, `tsc --noEmit` et `build`. La logique critique (heuristique de récence, multi-zones, cache) n'a aucune couverture — inacceptable pour un outil destiné à être vendu.

OBJECTIF

Couvrir la logique métier sensible par des tests automatisés exécutés en CI.

TÂCHES

- Mettre en place un runner de tests (ex. Vitest/Jest) côté backend.
- Tests unitaires sur `detectRecentContent`, `resolveWebsiteStatus`, la logique de sous-centres et le cache.
- Test d'intégration sur `/api/search` avec Google mocké.
- Brancher l'exécution des tests et la couverture dans la CI GitHub Actions.

CRITÈRES D'ACCEPTATION

- ✓ La CI exécute les tests à chaque PR et échoue en cas de régression.
- ✓ La logique de récence, de cache et de multi-zones est couverte.

RÉFÉRENCES : `.github/workflows/ci.yml` · `backend/src/services/websiteChecker.ts`

PW-26 Corriger la documentation d'installation (.env.example & README)

Majeur

CONSTAT

backend/.env.example ne contient que PORT et FRONTEND_URL ; il manque GOOGLE_PLACES_KEY et BACKEND_SECRET, pourtant requis (places.ts:172, auth.ts:4) — un développeur qui suit ce fichier obtient un backend cassé (503). Le README aggrave le problème : il liste BACKEND_SECRET à configurer sur Vercel uniquement, alors que auth.ts:4-9 le lit côté backend et renvoie 503 s'il est absent. Le README annonce aussi « Next.js 16.2.6 » quand package.json est en next ^15.3.3, et déclare une variable fantôme OVERPASS_API_URL.

OBJECTIF

Qu'un nouveau développeur puisse installer et démarrer le projet en suivant la documentation à la lettre.

TÂCHES

- Compléter backend/.env.example avec GOOGLE_PLACES_KEY et BACKEND_SECRET.
- Corriger la table d'environnement du README (BACKEND_SECRET + GOOGLE_PLACES_KEY côté Railway).
- Aligner les versions annoncées (README) sur package.json.
- Retirer la variable fantôme OVERPASS_API_URL du .env.example racine.

CRITÈRES D'ACCEPTATION

- ✓ Une installation suivant la doc produit un backend fonctionnel (pas de 503).
- ✓ Les versions et variables documentées correspondent au code.

RÉFÉRENCES : backend/.env.example · README.md · .env.example

PW-27 Supprimer le code mort et divergent

Mineur

CONSTAT

Plusieurs vestiges traînent dans le dépôt. lib/mock.ts (MOCK_ESTABLISHMENTS) n'est importé nulle part (12 fiches fictives résiduelles). ThemeProvider.tsx n'est pas importé et définit un schéma de thèmes divergent ('light'|'beige'|'dark'|'darker') alors que le thème réel utilise 'sable'|'violet'|'minuit'|'cosmos' via un cookie — source de confusion de maintenance.

OBJECTIF

Réduire la dette et la confusion en supprimant le code non utilisé et incohérent.

TÂCHES

- Supprimer frontend/app/lib/mock.ts après vérification qu'il n'est référencé nulle part.
- Supprimer frontend/app/components/ThemeProvider.tsx (non importé, schéma divergent).
- Vérifier qu'aucune régression de thème n'en découle.

CRITÈRES D'ACCEPTATION

- ✓ Le build et le lint passent après suppression.
- ✓ Aucun composant retiré n'était effectivement utilisé.

RÉFÉRENCES : frontend/app/lib/mock.ts · frontend/app/components/ThemeProvider.tsx

Au-delà de la viabilité : réduire structurellement le coût, sécuriser la continuité fournisseurs, construire un différenciateur défendable et poser les fondations d'un SaaS multi-tenant.

Majeur ×4

PW-28 Réduire le `FieldMask` Google au strict nécessaire par vue

Majeur

CONSTAT

Le `FieldMask` demandé (`places.ts:5-16`) inclut `websiteUri`, `nationalPhoneNumber`, `rating` et `userRatingCount` : ces champs « contact » et « atmosphère » font basculer chaque Nearby Search dans un palier tarifaire élevé, plutôt que dans le tier « IDs only » bon marché. Combiné aux multiplicateurs structurels (5 sous-centres par type), c'est un levier de coût direct.

OBJECTIF

Ne demander à Google que les champs réellement nécessaires à chaque étape, pour minimiser le palier tarifaire.

TÂCHES

- Cartographier les champs réellement nécessaires par vue (liste vs détail).
- Restreindre le `FieldMask` de la recherche initiale au minimum (idéalement IDs + champs de base).
- Charger les champs « contact » plus coûteux uniquement à l'ouverture d'une fiche, à la demande.
- Mesurer l'effet sur le coût par recherche.

CRITÈRES D'ACCEPTATION

- ✓ La recherche initiale n'inclut plus de champs coûteux non indispensables.
- ✓ Le coût par recherche mesuré diminue de façon vérifiable.

RÉFÉRENCES : `backend/src/services/places.ts:5-16`

PW-29 Plan de continuité fournisseurs (alerting, repli, circuit-breaker)

Majeur

CONSTAT

La dépendance à Google (Places + Geocoding), Nominatim, tuiles OSM, Vercel et Railway est dure et sans plan de continuité : une seule clé Google ; si elle est bloquée, en quota ou en facturation suspendue, [places.ts](#) renvoie des listes vides — le produit « marche » mais ne retourne plus rien, sans alerte. Aucun fournisseur de repli ni circuit-breaker.

OBJECTIF

Détecter les pannes fournisseurs, alerter, et dégrader proprement plutôt que silencieusement.

TÂCHES

- Alerting sur les erreurs Google (s'appuie sur PW-05) à destination de l'exploitant.
- Introduire un circuit-breaker pour éviter de marteler un fournisseur en panne.
- Abstraire le géocodeur et la source de places pour permettre un fournisseur de repli.
- Self-héberger ou proxifier Nominatim de manière identifiée pour le volume (lien avec PW-12).

CRITÈRES D'ACCEPTATION

- ✓ Une panne fournisseur déclenche une alerte exploitant.
- ✓ Une abstraction permet de basculer vers un fournisseur de repli.

RÉFÉRENCES : [backend/src/services/places.ts](#) · [backend/src/services/geocode.ts](#)

PW-30 Construire un différenciateur produit défendable

Majeur

CONSTAT

La proposition (Google Places → filtrer les « sans site » → mini-CRM → export) est reproductible en quelques jours. La seule « IP » est l'heuristique de récence, justement la plus fragile. Il n'y a pas de donnée propriétaire, pas d'effet de réseau, pas d'intégration verrouillante — la barrière à l'entrée est faible. Manquent aussi la déduplication inter-recherches, l'historique, et l'enrichissement (e-mail, dirigeant, SIRET).

OBJECTIF

Créer de la valeur difficilement copiable : enrichissement propriétaire, mémoire des recherches, intégrations.

TÂCHES

- Ajouter la déduplication inter-recherches et un historique de recherche.
- Enrichir les prospects (e-mail, dirigeant, SIRET) via des sources licites — ce qui alimente aussi PW-17.
- Évaluer des intégrations CRM externes verrouillantes.
- Ajouter un suivi de campagne / relances datées au CRM (au-delà des 5 statuts actuels).

CRITÈRES D'ACCEPTATION

- ✓ Les recherches voisines ne re-présentent plus les mêmes prospects sans contexte.
- ✓ Au moins une source d'enrichissement propriétaire/licite est intégrée.

RÉFÉRENCES : [frontend/app/hooks/useProspects.ts](#) · [frontend/app/prospects/components/ProspectsClient.tsx](#)

CONSTAT

En l'état : pas de comptes, pas d'auth, pas de facturation, pas de quota par client, pas de persistance serveur. L'outil ne peut pas être vendu en SaaS multi-tenant sans une reconstruction substantielle du backend. Aujourd'hui, c'est un outil interne mono-utilisateur — ce qu'il fait correctement.

OBJECTIF

Réunir les briques manquantes pour une offre SaaS multi-tenant : isolation, facturation, quotas, répercussion des coûts.

TÂCHES

- Isoler les données par tenant/utilisateur (s'appuie sur PW-07, PW-14).
- Intégrer une facturation et des plans (avec quotas par plan, lien avec PW-08).
- Répercuter le coût Google par usage / par plan.
- Définir les rôles et le partage d'équipe si pertinent.

CRITÈRES D'ACCEPTATION

- ✓ Plusieurs clients peuvent utiliser le produit avec des données strictement isolées.
- ✓ La facturation et les quotas par plan sont opérationnels.

RÉFÉRENCES : [Architecture backend \(transverse\)](#)

Cette feuille de route traduit fidèlement les constats et recommandations de l'audit confidentiel du 22 juin 2026. Les criticités reprennent celles attribuées par l'auditeur. Les références [fichier:ligne](#) ont été recoupées avec l'état du dépôt [sofianehms/prospectweb](#). L'ordre des milestones (M0 → M6) reflète le séquençage conseillé : neutraliser d'abord les risques financiers et juridiques et les données fabriquées (M0), puis reconstruire le socle sécurité/persistance (M1-M2), fiabiliser le produit et le mettre en conformité (M3-M4), enfin consolider qualité, scalabilité et différenciation (M5-M6).